

ECE 3301 — Lab 8: Drone Altitude & LCD Flight Dashboard

ECE 3301 – Microcontrollers | Fall 2026

Instructor: Sasoun Torousian

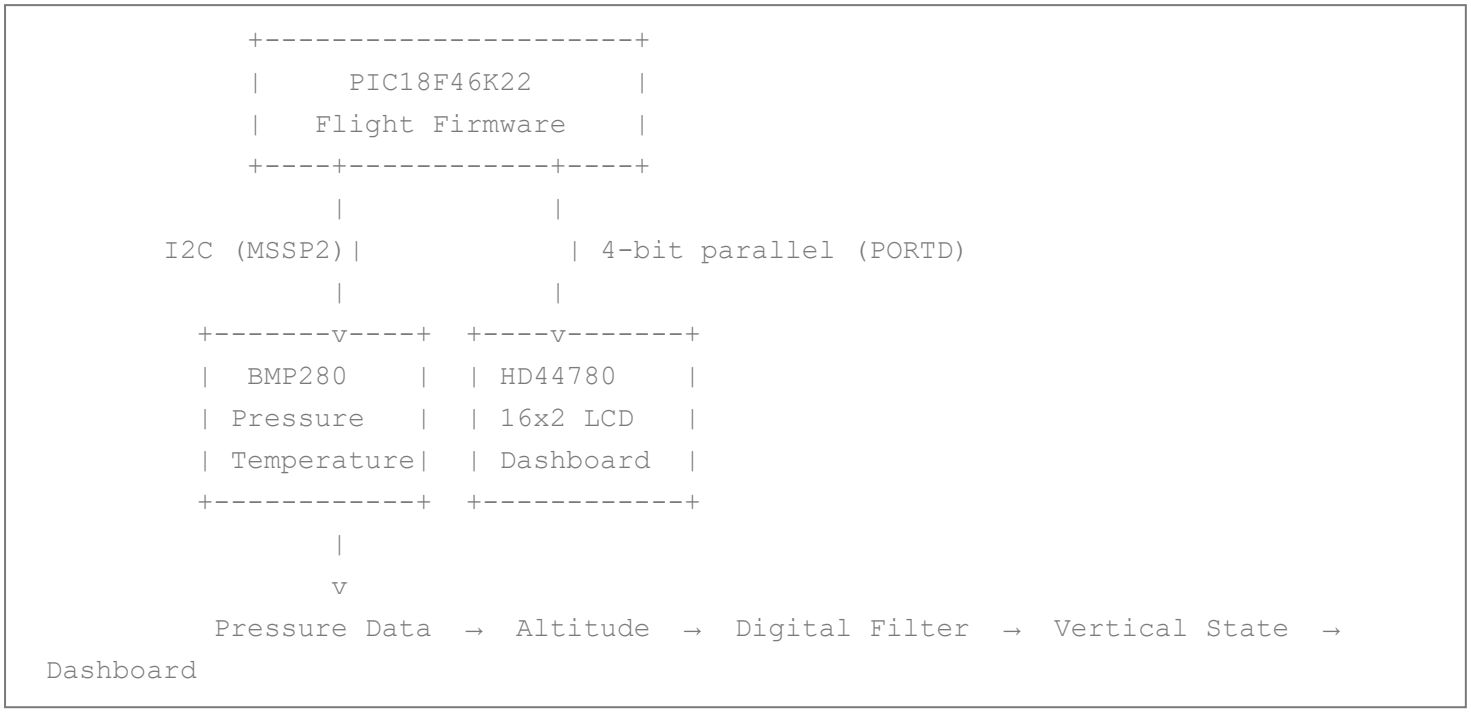
Introduction

In the previous labs you built the firmware architecture of a small drone flight controller. You developed timing and safety logic, converted important events to interrupts, and added the ICM-20948 IMU over SPI. The IMU tells the flight controller about motion, acceleration and rotation.

A real drone also needs to know whether it is climbing, descending or holding altitude. A common way to get this is a barometric pressure sensor.

In this lab you will:

- add the BMP280 pressure/temperature sensor to the PIC18F46K22 using I²C;
- convert pressure into relative altitude;
- filter the measurement and classify the vertical flight state;
- add an HD44780 16×2 LCD as the first onboard flight dashboard.



The key engineering idea: the microcontroller is no longer doing one job. It must talk to a sensor, compute altitude, filter noise, classify motion, update a display and keep the safety architecture running, without stalling the processor in long delays.

Learning Objectives

After this lab you should be able to:

- configure an MSSP module of the PIC18F46K22 as an I²C master;
- read and write BMP280 registers, including multi-byte burst reads;

- verify the identity of an I²C peripheral before trusting its data;
- read factory calibration coefficients and apply the compensation equations;
- convert pressure into relative altitude using a locally measured reference;
- design a low-pass filter and relate its coefficient to a time constant;
- classify vertical motion using a deadband with hysteresis;
- integrate a provided device driver (HD44780 LCD) and explain its timing;
- run several tasks at different rates without blocking delays;
- detect and report a sensor fault instead of hanging.

Parts Required

Qty	Part	Notes
1	Microchip Curiosity HPC Development Board (DM164136)	Power jumper **J45 on 3V3** (see Section 4)
1	PIC18F46K22	40-pin DIP in the **40-pin socket**. The 28-pin socket must be empty (Section 5.4)
1	HiLetgo BMP280 3.3 V module	I²C mode
1	HD44780-compatible 16×2 character LCD	Parallel interface, 16-pin header
1	10 kΩ potentiometer	LCD contrast (V0)
2	4.7 kΩ resistors	I²C pull-ups, **only if** the BMP280 module has none
—	Breadboard, jumper wires, USB cable	

---

1. What You Are Building

A simplified barometric altitude subsystem.

At startup, the system verifies that the BMP280 is present, then measures the local pressure while the sensor sits still. That pressure becomes the reference for 0 m. Afterwards, changes in pressure are converted to relative altitude, filtered, and classified into one of four vertical states:

State	Meaning
`GROUND`	Stationary, near the reference altitude (on the bench)
`CLIMB`	Altitude increasing faster than the climb threshold
`HOLD`	Stationary, away from the reference altitude
`DESCEND`	Altitude decreasing faster than the descend threshold

The LCD shows two dashboard pages that alternate automatically:

```
Page 1                Page 2
+----------------+    +----------------+
|ALT:   1.24 m   |    |P: 97845.2 Pa   |
|STATE: CLIMB    |    |T: 24.6 C       |
+----------------+    +----------------+
```

You do not need these exact characters, but the information must be clear and readable.

---

2. Why Pressure Can Measure Altitude

Atmospheric pressure decreases as altitude increases. Standard sea-level pressure is approximately:

$$P_{SL} \approx 101325 \text{ Pa}$$

A climbing drone sees pressure decrease. A descending drone sees pressure increase.

A commonly used barometric altitude equation is:

$$h = 44330 \left[1 - \left(\frac{P}{P_{ref}} \right)^{0.1903} \right]$$

Symbol	Meaning
h	altitude above the reference point, in meters
P	current pressure (Pa)
P_{ref}	reference pressure measured at startup (Pa)

How sensitive is this?

Differentiating near $P = P_{ref}$:

$$\frac{dh}{dP} \approx -\frac{44330 \times 0.1903}{P_{ref}} \approx -0.083 \text{ m/Pa}$$

One pascal of pressure change is about 8 cm of altitude. Keep this number in mind. It explains why a few pascals of sensor noise look like the drone moving up and down.

---

3. Reference Pressure

Using 101325 Pa as the reference would be a poor design. Pressure varies with weather and with the elevation of the laboratory, so your bench would not read 0 m.

Instead, your system measures its own reference. While the sensor is stationary on the bench:

$$P_{ref} = \frac{P_1 + P_2 + \cdots + P_N}{N}$$

Use N = 32 samples, after discarding the first few readings (see Section 14).

> *Float precision tip. The PIC uses 32-bit* `float` *(about 7 significant digits). A sum of 32 pressures is about 3,000,000 Pa, where float resolution is about 0.25 Pa, roughly 2 cm. A cleaner method is to accumulate the difference from the first sample:* `sum += (p - pFirst);` *then* `Pref = pFirst + sum / N;`

---

4. Power and Voltage Plan — Read Before Wiring

This lab mixes a 3.3 V sensor with an LCD that is usually a 5 V device. Get this right before applying power.

4.1 Run the PIC at 3.3 V

The Curiosity HPC has three supply rails available on its headers:

Rail	Label on board	Source	Depends on J45?
PIC supply	**TVDD**	Selected by jumper J45	**Yes**

Rail	Label on board	Source	Depends on J45?
Fixed 3.3 V	**3V3**	On-board 3.3 V regulator	No — always 3.3 V
Fixed 5 V	**5V**	USB 5 V through a current-limited switch	No — always 5 V

J45 is the 3-pin jumper in the top-left corner, next to the RESET button, with silkscreen 5V above and 3V3 below. Place the shunt on the 3V3 side.

With the board powered, measure the TVDD test pad (next to the GND pad, to the right of LEDs D2–D5). It must read about 3.3 V before you connect anything.

Why: the BMP280 is a 3.3 V part, and its I²C pull-ups go to 3.3 V. If the PIC ran at 5 V, its I²C input-high threshold would be about 0.7 × 5 V = 3.5 V. A 3.3 V logic high might not register reliably. Running everything on the I²C bus at 3.3 V removes the problem.

4.2 Power the LCD from 5 V, write-only

Most HD44780 modules need 5 V for usable contrast.

- LCD VDD → a 5V header pin (next to the mikroBUS sockets), LCD VSS → GND. This rail stays at 5 V regardless of J45.
- The PIC drives the LCD inputs at 3.3 V. That is valid: the HD44780U datasheet specifies an input-high threshold of 2.2 V minimum when the LCD runs at 4.5–5.5 V. (At 2.7–4.5 V the threshold becomes 0.7 × VCC, which is why a 5 V LCD is easier to drive from 3.3 V than it looks.)
- The LCD has weak internal pull-ups (roughly 50–250 µA at 5 V) on RS and DB0–DB7. When the PIC drives these pins high at 3.3 V, a small current flows back into the PIC's 3.3 V supply. With six lines this is under about 1.5 mA, which is harmless.
- 5 V also gives proper contrast. The LCD needs VCC - V0 of at least 3 V; from a 5 V supply the contrast pot has plenty of range, while from 3.3 V it barely does.
- LCD R/W → GND, permanently. The LCD is only ever written. If R/W were high, the LCD would drive 5 V back onto PIC pins that are powered at 3.3 V and are not 5 V tolerant. That can damage the PIC.
- Because R/W is grounded, the busy flag cannot be read. The provided LCD driver uses fixed delays instead (Section 13.5).

If your LCD module is specifically rated for 3.3 V, power it from 3.3 V instead. R/W still goes to GND.

4.3 I²C pull-ups

I²C is open-drain. The bus high level comes only from pull-up resistors, so their supply voltage is the bus voltage.

- Most BMP280 breakout modules already have pull-ups to their 3.3 V supply. With power off, measure resistance from SDA to the module VCC and from SCL to VCC. A reading of a few kΩ to about 10 kΩ means pull-ups are present.
- If none are present, add 4.7 kΩ from SDA to a 3V3 pin and from SCL to a 3V3 pin.
- The Curiosity HPC provides no I²C pull-ups for the target PIC. The 4.7 kΩ pull-ups visible in the board schematic belong to the on-board debugger's own I²C EEPROM, not to your bus.
- Never pull SDA/SCL up to 5 V with a 3.3 V sensor on the bus.
- With power on and the bus idle, SDA and SCL should both read about 3.3 V.

4.4 Common ground

The PIC, BMP280 and LCD must share one ground.

---

5. Wiring

The ICM-20948 from the previous lab uses MSSP1 (SPI on RC3/RC4/RC5). An MSSP module cannot be SPI and I²C at the same time, so the BMP280 uses MSSP2, whose I²C pins on the 40-pin PIC18F46K22 are RD0 (SCL2) and RD1 (SDA2). The LCD uses the rest of PORTD.

5.1 BMP280

BMP280 pin	Connect to	Where on the Curiosity HPC	Notes
VCC	**3V3**	3V3 pin beside the mikroBUS sockets	Fixed regulator output. The sensor stays at 3.3 V even if J45 is set wrong
GND	GND	Any GND pin	
SCL	**RD0 / SCL2**	Bottom header (J8), second-to-last pin, labeled RD0	
SDA	**RD1 / SDA2**	Bottom header (J8), last pin, labeled RD1	

BMP280 pin	Connect to	Where on the Curiosity HPC	Notes
CSB	3V3 (or leave if pulled up on module)		Must be high at power-up, or the chip latches into SPI mode
SDO	GND		Selects I²C address **0x76** (SDO high → 0x77)

5.2 HD44780 LCD (4-bit mode)

LCD pin	Name	Connect to	Where on the Curiosity HPC
1	VSS	GND	Any GND pin
2	VDD	**5V** (see Section 4.2)	5V pin beside the mikroBUS sockets
3	V0	Wiper of 10 kΩ pot (pot ends to 5V and GND)	Breadboard
4	RS	**RD2**	Top header (J11), last pin, labeled RD2
5	R/W	**GND**	
6	E	**RD3**	Top header (J11), labeled RD3
7–10	D0–D3	Not connected	
11	D4	**RD4**	Top header (J11), labeled RD4
12	D5	**RD5**	Top header (J11), labeled RD5
13	D6	**RD6**	Top header (J11), labeled RD6
14	D7	**RD7**	Top header (J11), labeled RD7

LCD pin	Name	Connect to	Where on the Curiosity HPC
15	A (backlight +)	5V, through a series resistor if the module has none	
16	K (backlight −)	GND	

Every header pin on the Curiosity HPC is labeled with its port name on the silkscreen. Use the labels, not pin counting.

5.3 Port configuration

PORTD pins have analog capability on the PIC18F46K22, and they power up as analog inputs. Every PORTD pin you use must be made digital, or the I²C and LCD pins will not work.

The work is split between two pieces of code:

Pins	Who configures them	What is set
RD2–RD7 (LCD)	`LCD_Init()` in the provided driver	`ANSELD` bits 2–7 = 0, `TRISD` bits 2–7 = 0 (outputs)
RD0–RD1 (I²C)	**Your** `I2C_Init()`	`ANSELD` bits 0–1 = 0, `TRISD` bits 0–1 = 1 (inputs; MSSP2 drives them)

Use bit operations so each piece of code touches only its own pins:

```
// In your I2C_Init(), before enabling MSSP2:
ANSELD &= ~0x03;    // RD0, RD1 digital
TRISD  |=  0x03;    // RD0, RD1 inputs - required for I2C
```

Do not write `TRISD = ...` or `ANSELD = ...` as a whole byte after `LCD_Init()` has run, or you will undo the LCD configuration.

5.4 Curiosity HPC board conflicts

The board routes some PIC pins to its own peripherals. For this lab:

Board feature	PIC pins	Effect on this lab
mikroBUS socket 1	RST → **RD0**; SCL/SDA → **RC3/RC4**	RD0 is your SCL2. RC3/RC4 are MSSP1 (your IMU SPI).
mikroBUS socket 2	PWM → **RD1**, RST → **RD2**, CS → **RD3**; SCL/SDA → **RC3/RC4**	RD1 is your SDA2; RD2/RD3 are LCD RS/E.
LEDs D2–D5	RA4–RA7	Not used by Lab 8 pins
Push button S1	RB4	Not used by Lab 8 pins
Push button S2	RC5	Shares the pin with SDO1 (IMU SPI). Do not press S2 while the IMU is running
Potentiometer	RA0	Not used by Lab 8 pins
On-board programmer	RB6/RB7, MCLR	Leave unconnected

Rules that follow from this:

- Keep both mikroBUS sockets empty. An empty socket is just an open connector. A Click board plugged into either socket would load your I²C, LCD and SPI lines.
- Use only one PIC. The 28-pin and 40-pin sockets are wired to the same nets. The board ships with a PIC18F47Q10 in the 40-pin socket; replace it with your PIC18F46K22 and leave the 28-pin socket empty.
- Keep LVP enabled. The on-board programmer only uses low-voltage programming, so the configuration bits must include `LVP = ON`.

---

6. I²C Configuration (MSSP2 Master, 100 kHz)

```
void I2C_Init(void)
{
    SSP2STAT = 0x80;                            // SMP = 1: slew-rate control off
(100 kHz)
    SSP2ADD  = (_XTAL_FREQ / (4UL * 100000UL)) - 1;   // baud-rate generator
    SSP2CON1 = 0x28;                            // SSPEN = 1, I2C Master mode
    SSP2CON2 = 0x00;
}
```

Fosc	SSP2ADD for 100 kHz
16 MHz	39
64 MHz	159

6.1 Every wait must have a timeout

If a wire falls out or the bus locks up, a loop like `while (SSP2CON2bits.SEN);` waits forever. Your entire flight controller, including safety logic, would freeze. Every I²C wait loop must give up after a bounded number of iterations and report an error.

Example idle-wait pattern:

```
#define I2C_TIMEOUT_COUNT  10000u

// Returns 1 when the bus is idle, 0 on timeout.
static uint8_t I2C_WaitIdle(void)
{
    uint16_t t = I2C_TIMEOUT_COUNT;

    // SSP2CON2 bits 0-4 (SEN, RSEN, PEN, RCEN, ACKEN) and SSP2STAT R/W (bit 2)
    while ((SSP2CON2 & 0x1F) || (SSP2STAT & 0x04))
    {
        if (--t == 0) return 0;
    }
    return 1;
}
```

Your I²C functions (`Start`, `RepeatedStart`, `Stop`, `Write`, `Read`) should return a status so the BMP280 driver can detect failure. If a write is not ACKed (`SSP2CON2bits.ACKSTAT == 1`), treat it as a failure as well.

---

7. BMP280 Register Summary

Register	Address	Use
Calibration `dig_T1` … `dig_P9`	0x88 – 0x9F	24 bytes, **little-endian** (LSB first)
`id`	0xD0	Reads **0x58** for BMP280
`reset`	0xE0	Write **0xB6** for soft reset

Register	Address	Use
`status`	0xF3	bit 3 `measuring`, bit 0 `im_update`
`ctrl_meas`	0xF4	`osrs_t[7:5]` `osrs_p[4:2]` `mode[1:0]`
`config`	0xF5	`t_sb[7:5]` `filter[4:2]` `spi3w_en[0]`
`press_msb/lsb/xlsb`	0xF7 – 0xF9	20-bit raw pressure, **MSB first**
`temp_msb/lsb/xlsb`	0xFA – 0xFC	20-bit raw temperature, **MSB first**

7.1 The address: 7-bit vs. 8-bit

The datasheet gives a 7-bit address. The PIC sends a byte containing the address shifted left plus the R/W bit. Define it once, clearly:

```
#define BMP280_ADDR7   0x76                    // SDO tied to GND
#define BMP280_WRITE   ((BMP280_ADDR7 << 1) | 0) // 0xEC
#define BMP280_READ    ((BMP280_ADDR7 << 1) | 1) // 0xED
#define BMP280_CHIP_ID 0x58
```

Do not assume. If ID reads fail at 0x76, check SDO wiring and try 0x77.

---

8. First Requirement — Verify I²C Communication

Do not begin by calculating altitude. First prove the electrical connection and I²C transaction work.

Read the ID register:

```
START
  ↓
Send BMP280_WRITE (0xEC)        → must be ACKed
  ↓
Send register address 0xD0      → must be ACKed
  ↓
REPEATED START
  ↓
Send BMP280_READ (0xED)         → must be ACKed
  ↓
Read 1 byte, send NACK
  ↓
STOP
```

Display the ID on the LCD (the LCD is brought up first; see Section 25).

ID read	Meaning
`0x58`	BMP280 — continue
`0x60`	BME280 (a mislabeled module). Communication works, and pressure/temperature registers are compatible. Tell your instructor.
`0xFF` or timeout	No device responding: wiring, pull-ups, power, or wrong address
`0x00`	SDA held low or a stuck bus

The debugging rule for this lab:

```
Do not debug Level 4 until Levels 1–3 work.

Level 4 → Altitude algorithm
Level 3 → Sensor data
Level 2 → I2C transactions
Level 1 → Power and wiring
```

---

9. Configure the BMP280

Use these settings. They are chosen to match the 50 ms sampling rate of this lab.

Setting	Value	Reason
Mode	Normal	Sensor converts continuously; firmware just reads the latest result
Pressure oversampling `osrs_p`	×16 (`101`)	Lowest pressure noise
Temperature oversampling `osrs_t`	×2 (`010`)	Sufficient for compensation
Standby `t_sb`	0.5 ms (`000`)	Fastest update
Internal IIR filter	**Off** (`000`)	You will build the filter yourself; see note below

Resulting register values:

```
#define BMP280_CONFIG_VALUE     0x00    // t_sb = 0.5 ms, filter off
#define BMP280_CTRL_MEAS_VALUE  0x57    // osrs_t x2, osrs_p x16, normal mode
```

With these settings, one conversion takes about 38 ms (typical) to 43 ms (maximum). The sensor produces a new result roughly every 40 ms, about 25 Hz. That is faster than your 20 Hz sampling, so every read returns a fresh measurement. If you chose a long standby time, you would read the same value twice in a row and your noise and Δh measurements would be wrong.

> *Why the internal IIR filter is off. If both the sensor's IIR filter and your firmware filter are active, their delays add, and the "raw" altitude you observe in Stage 9 would already be filtered. For this lab, you design the filter.*

9.1 Initialization sequence

```
uint8_t BMP280_Init(void)
{
    // 1. Read ID (0xD0). If not 0x58 (or 0x60), return failure.
    // 2. Soft reset: write 0xB6 to 0xE0. Wait ~2 ms (startup only).
    // 3. Wait until status.im_update (bit 0) = 0 — calibration copied to
registers.
    // 4. Burst-read 24 calibration bytes starting at 0x88.
    // 5. Write config (0xF5) FIRST, while the sensor is still in sleep mode.
    //    Writes to config in normal mode may be ignored.
    // 6. Write ctrl_meas (0xF4) = 0x57 to start normal mode.
    // Return success.
}
```

Do not copy a large sensor library that hides the communication. You may use the compensation equations from the BMP280 datasheet, reproduced in Section 11.

---

10. Read Calibration and Raw Data

10.1 Calibration coefficients

The 24 bytes at 0x88–0x9F hold 12 coefficients, each LSB first:

Coefficient	Address	Type
`dig_T1`	0x88/0x89	`uint16_t`

Coefficient	Address	Type
`dig_T2`, `dig_T3`	0x8A – 0x8D	`int16_t`
`dig_P1`	0x8E/0x8F	`uint16_t`
`dig_P2` … `dig_P9`	0x90 – 0x9F	`int16_t`

```
dig_T1 = (uint16_t)((buf[1] << 8) | buf[0]);   // note: LSB is the lower address
dig_T2 = (int16_t) ((buf[3] << 8) | buf[2]);
```

10.2 Raw data — one burst read

Read all six bytes 0xF7–0xFC in a single transaction. The BMP280 locks the data registers during a burst read, so pressure and temperature come from the same conversion. Separate transactions could mix two different conversions.

The data registers are MSB first, and the result is 20 bits:

```
adc_P = ((uint32_t)b[0] << 12) | ((uint32_t)b[1] << 4) | (b[2] >> 4);
adc_T = ((uint32_t)b[3] << 12) | ((uint32_t)b[4] << 4) | (b[5] >> 4);
```

> *8-bit trap. On XC8, `int` is 16 bits. Without the `(uint32_t)` cast, `b[0] << 12` overflows silently and your pressure will be garbage. This is the most common bug in this lab.*

---

11. Compensation — Temperature First

The BMP280 output must be corrected using the calibration coefficients. Temperature must be compensated first, because the pressure equation uses `t_fine`, an intermediate value produced by the temperature equation.

Required order on every sample:

```
Burst-read 6 raw bytes  →  compensate temperature (produces t_fine)  →  compensate pressure
```

Floating-point compensation (from the BMP280 datasheet):

```
static float t_fine;

float BMP280_CompensateTemperature(int32_t adc_T)
```

```
{
    float var1, var2;
    var1 = ((float)adc_T / 16384.0f - (float)dig_T1 / 1024.0f) * (float)dig_T2;
    var2 = ((float)adc_T / 131072.0f - (float)dig_T1 / 8192.0f);
    var2 = var2 * var2 * (float)dig_T3;
    t_fine = var1 + var2;
    return t_fine / 5120.0f;                    // degrees C
}

float BMP280_CompensatePressure(int32_t adc_P)  // call AFTER temperature
{
    float var1, var2, p;
    var1 = t_fine / 2.0f - 64000.0f;
    var2 = var1 * var1 * (float)dig_P6 / 32768.0f;
    var2 = var2 + var1 * (float)dig_P5 * 2.0f;
    var2 = var2 / 4.0f + (float)dig_P4 * 65536.0f;
    var1 = ((float)dig_P3 * var1 * var1 / 524288.0f + (float)dig_P2 * var1) /
524288.0f;
    var1 = (1.0f + var1 / 32768.0f) * (float)dig_P1;
    if (var1 == 0.0f) return 0.0f;              // avoid divide-by-zero
    p = 1048576.0f - (float)adc_P;
    p = (p - var2 / 4096.0f) * 6250.0f / var1;
    var1 = (float)dig_P9 * p * p / 2147483648.0f;
    var2 = p * (float)dig_P8 / 32768.0f;
    return p + (var1 + var2 + (float)dig_P7) / 16.0f;   // pascals
}
```

Compiler setting: in the MPLAB X project properties for XC8, set both `float` and `double` to 32-bit. A 24-bit float has only about 16 bits of mantissa, which near 100,000 Pa limits resolution to about 1.5 Pa, roughly 13 cm of altitude.

11.1 Sanity check every sample

Reject physically impossible values and count them as a sensor fault:

- Pressure must be between 30,000 Pa and 110,000 Pa.
- Temperature must be between -40 °C and 85 °C.

Suggested driver API:

```
uint8_t BMP280_Init(void);                  // returns 1 on success
uint8_t BMP280_ReadRegister(uint8_t reg, uint8_t *value);
uint8_t BMP280_ReadCalibration(void);
uint8_t BMP280_ReadRaw(void);               // one 6-byte burst read
float   BMP280_GetTemperature(void);        // uses latest raw data, sets t_fine
float   BMP280_GetPressure(void);           // uses latest raw data and t_fine
```

Function names may differ, but the program must be logically separated, and the order ReadRaw → Temperature → Pressure must be preserved.

---

12. Calculate Relative Altitude

$$h = 44330 \left[1 - \left(\frac{P}{P_{ref}} \right)^{0.1903} \right]$$

```
float CalculateAltitude(float p, float pRef)
{
    return 44330.0f * (1.0f - powf(p / pRef, 0.1903f));
}
```

Do not update P_{ref} after calibration. If the reference kept following the current pressure, altitude would always drift back toward zero and the system would lose its reference.

---

13. LCD Driver (Provided)

You are given a tested driver for the HD44780 LCD: `lcd.h` and `lcd.c`. The full listing is in Appendix A, and the files are posted on Canvas.

You do not write the low-level LCD code in this lab. You use the driver, and you must be able to explain how it works (Section 13.5). Do not modify `lcd.c`. If you believe it has a bug, send me an email, this is the first time I am using this driver.

13.1 Adding the driver to your project

1. Copy `lcd.h` and `lcd.c` into your MPLAB X project folder.
2. In the Projects window:
    - right-click Header Files → Add Existing Item... and select `lcd.h`;
    - right-click Source Files → Add Existing Item... and select `lcd.c`.
3. Define the CPU clock for the whole project:
   Project Properties → XC8 Compiler → Preprocessing and messages → Define macros, add `_XTAL_FREQ=16000000` (use your actual clock frequency in Hz).
4. Remove any `#define _XTAL_FREQ` line from `main.c`. The project-wide macro now provides it to every file.
5. At the top of `main.c`:

```
#include "lcd.h"
```

> *Why step 3 matters.* `__delay_us()` *and* `__delay_ms()` *calculate their delays from* `_XTAL_FREQ`*. A* `#define` *inside* `main.c` *is invisible to* `lcd.c`*, so* `lcd.c` *would not compile. If* `_XTAL_FREQ` *is lower than your real clock, every LCD delay becomes too short and the display fails to initialize. If it is missing entirely,* `lcd.c` *stops the build with an error that tells you what to do.*

13.2 Driver functions

Function	What it does	Blocking time
`LCD_Init()`	Configures RD2–RD7 and initializes the LCD. Call **once** at startup.	~60 ms
`LCD_PrintLine(row, str)`	Writes a full line: exactly 16 characters, padded with spaces or truncated. **Use this for the dashboard.**	~1.1 ms
`LCD_SetCursor(row, col)`	Moves the cursor. `row` 0 = top, 1 = bottom; `col` 0–15.	~0.06 ms
`LCD_Print(str)`	Writes a string at the cursor, no padding.	~0.06 ms per character
`LCD_Char(c)`	Writes one character at the cursor.	~0.06 ms
`LCD_Clear()`	Clears the whole display. **Do not use in periodic updates.**	3 ms
`LCD_Command(cmd)`	Sends a raw HD44780 instruction. You should not need it.	0.06–3 ms

13.3 Using it in the dashboard

Format each line into a buffer with `snprintf`, then write it with `LCD_PrintLine`:

```
#include <stdio.h>
#include "lcd.h"

static const char *StateName(VerticalState s)
{
    switch (s)
    {
```

```
        case GROUND:  return "GROUND";
        case CLIMB:   return "CLIMB";
        case HOLD:    return "HOLD";
        case DESCEND: return "DESCEND";
        default:      return "?";
    }
}

void UpdateFlightDashboard(void)        // called from the main loop every 250 ms
{
    char line[17];                      // 16 characters + terminating '\0'

    snprintf(line, sizeof line, "ALT:%7.2f m", filteredAltitude);
    LCD_PrintLine(0, line);

    snprintf(line, sizeof line, "STATE: %s", StateName(verticalState));
    LCD_PrintLine(1, line);
}
```

At startup, before the scheduler runs:

```
LCD_Init();
LCD_PrintLine(0, "ECE3301 LAB 8");
LCD_PrintLine(1, "STARTING...");
```

Notes:

- `LCD_PrintLine` pads short strings with spaces, so a shorter value never leaves old characters behind. You do not need to add trailing spaces yourself.
- `%f` handles negative values correctly, e.g. `ALT:  -0.05 m`. If you format numbers with integer arithmetic instead, handle the sign yourself: splitting -0.05 naively prints `0.-5`.
- The buffer must be at least 17 bytes. `snprintf` with `sizeof line` can never overflow it.

13.4 Rules for using the driver

- Main loop only. Never call an LCD function from an ISR. The driver uses busy-wait delays.
- Never in a periodic task: `LCD_Clear()`. It takes 3 ms and makes the display flicker. Overwrite lines with `LCD_PrintLine` instead.
- Blocking budget. One dashboard update (two lines) blocks for about 2.2 ms every 250 ms, under 1% of CPU time. That is acceptable because it is short and bounded. `LCD_Init()` blocks about 60 ms, which is acceptable once at startup.
- Wiring is fixed. RS = RD2, E = RD3, D4–D7 = RD4–RD7, R/W = GND. The driver never touches RD0/RD1, so `LCD_Init()` and `I2C_Init()` can be called in either order.

13.5 How the driver works — you must be able to explain this

Write-only interface. R/W is tied to GND (Section 4.2), so the driver can never read the LCD's busy flag. Instead, it waits a fixed time after every instruction, long enough for the slowest LCD the datasheet allows.

Operation	Datasheet value (5 V)	Driver uses
Power-up before first command	> 15 ms after VCC reaches 4.5 V	50 ms
E pulse high width	≥ 230 ns	1 µs
RS stable before E rises	≥ 40 ns	RS is set first
Data stable before E falls	≥ 80 ns	Data is set before E rises
Most instructions and characters	37 µs at 270 kHz	**60 µs**
Clear Display (0x01), Return Home (0x02)	1.52 ms at 270 kHz	**3 ms**

Why 60 µs and not 37 µs? The datasheet times assume the LCD's internal oscillator runs at 270 kHz, but it is only guaranteed between 190 and 350 kHz. At 190 kHz every instruction takes about 1.4× longer: 37 µs becomes about 53 µs, and 1.52 ms becomes about 2.2 ms. The driver's delays cover that worst case.

Sending one byte in 4-bit mode. The LCD latches data on the falling edge of E. Each byte is sent as two nibbles, high nibble first:

1. Set RS (0 = instruction, 1 = character).
2. Put the nibble on RD4–RD7, leaving RD0–RD3 unchanged.
3. E = 1, wait 1 µs.
4. E = 0, wait 1 µs. *(The LCD latches the nibble here.)*
5. Repeat steps 2–4 for the low nibble, then wait the execution time.

Initialization sequence. The driver uses the datasheet's *initializing by instruction* procedure, because the LCD's automatic power-on reset only works if the 5 V supply rises in 0.1–10 ms, which a USB-powered board does not guarantee:

```
Wait 50 ms after power-up
Send nibble 0x3, wait 5 ms      \
Send nibble 0x3, wait 150 µs     |  forces 8-bit mode from any unknown state
Send nibble 0x3, wait 150 µs    /
Send nibble 0x2, wait 150 µs    → switch to 4-bit mode
Command 0x28                    → 4-bit, 2 lines, 5x8 font
Command 0x08                    → display off
Command 0x01, wait 3 ms         → clear
Command 0x06                    → entry mode: increment, no shift
Command 0x0C                    → display on, cursor off
```

DDRAM addresses. On a 2-line display, line 1 starts at address 0x00 and line 2 at 0x40. `LCD_SetCursor(row, col)` sends `0x80 | (base + col)`, so the top-left corner is command 0x80 and the bottom-left is 0xC0.

---

14. System States and Non-Blocking Calibration

Calibration is not a blocking loop before the scheduler starts. It is a system state driven by the sensor task, so the timer, safety events and LCD keep running throughout.

```
typedef enum
{
    SYS_CALIBRATING,
    SYS_RUNNING,
    SYS_SENSOR_FAULT
} SystemState;
```

14.1 Calibration procedure

Each `sensorTask` tick (every 50 ms) while in `SYS_CALIBRATING`:

1. Read the sensor (ReadRaw → Temperature → Pressure).
2. Discard the first 5 samples (`N_DISCARD`) so the sensor output has settled.
3. Accumulate the next 32 samples (`N_CAL`), using the difference-from-first-sample method in Section 3. Track the minimum and maximum pressure.
4. When 32 samples are collected:
    - If `max - min > CAL_MAX_SPREAD_PA`, the sensor moved. Restart calibration.
    - Otherwise compute P_{ref}, set `filteredAltitude = 0`, clear the Δh history (Section 16), set `verticalState = GROUND`, and switch to `SYS_RUNNING`.

Total calibration time ≈ (5 + 32) × 50 ms ≈ 1.9 s.

Choose `CAL_MAX_SPREAD_PA` from your stationary noise measurement. Start around 5 Pa.

14.2 LCD during calibration

```
+----------------+
|CALIBRATING...  |
|KEEP STILL 12/32|
+----------------+
```

When finished:

```
+----------------+
|READY           |
|ALT:   0.00 m   |
+----------------+
```

Show "READY" for about 1 s, then switch to the normal dashboard.

---

15. Why Altitude Moves When the Sensor Does Not

Place the sensor on the bench and watch the raw altitude. It will not be constant.

```
 0.06 m
-0.04 m
 0.11 m
-0.08 m
 0.02 m
-0.11 m
 0.05 m
```

This does not mean your code is wrong. Real sensors have noise. The BMP280 pressure noise is on the order of 1 Pa, and from Section 2, 1 Pa ≈ 8 cm. Right after calibration, the raw values should scatter around zero, not sit consistently above or below it.

Other real effects you may see:

- Hand heat. Holding the module warms it.
- Air movement. Drafts, HVAC and doors change local pressure.
- Weather drift. Over minutes, the reading slowly wanders by tens of centimeters. This is real pressure change, not a bug.

If raw altitude were fed directly to the flight logic, the controller would conclude the drone is constantly climbing and descending. Real flight controllers filter sensor data.

---

16. Altitude Filtering

Implement a first-order low-pass (exponential moving average) filter:

$$h_f(k) = \alpha\, h_f(k-1) + (1-\alpha)\, h(k)$$

Symbol	Meaning
$h(k)$	newest raw altitude
$h_f(k)$	filtered altitude
$h_f(k-1)$	previous filtered altitude
α	filter coefficient, $0 \leq \alpha < 1$

```
filteredAltitude = alpha * filteredAltitude + (1.0f - alpha) * rawAltitude;
```

16.1 What α actually means

The filter's time constant depends on α and the sample period T:

$$\tau = \frac{-T}{\ln \alpha}$$

At $T = 50$ ms:

α	Time constant τ	Behavior
0.50	≈ 0.07 s	Fast, noisy
0.70	≈ 0.14 s	
0.80	≈ 0.22 s	
0.90	≈ 0.47 s	
0.95	≈ 0.98 s	Smooth, slow

After a step change, the filtered value reaches about 63% of the change after τ and about 95% after 3τ.

A larger α gives smoother data but slower response. A smaller α responds faster but passes more noise. If you change the sampling rate, the same α gives a different time constant.

Select α experimentally. Move the sensor and observe both stability and response time. Do not choose a value because it "looks good" in someone else's code.

---

17. Vertical Flight State

17.1 Measuring vertical motion over a window

Comparing two consecutive samples 100 ms apart does not work well. A slow hand lift of about 0.3 m/s moves only 3 cm in 100 ms, which is comparable to the noise. Instead, compare the current filtered altitude with the filtered altitude 500 ms earlier:

$$\Delta h = h_f(\text{now}) - h_f(\text{now} - 500\ \text{ms})$$

The state task runs every 100 ms, so keep a small ring buffer of the last 5 filtered altitudes:

```
#define DH_WINDOW_STEPS  5           // 5 x 100 ms = 500 ms window

static float   altHistory[DH_WINDOW_STEPS];   // cleared to 0 after calibration
static uint8_t histIndex = 0;

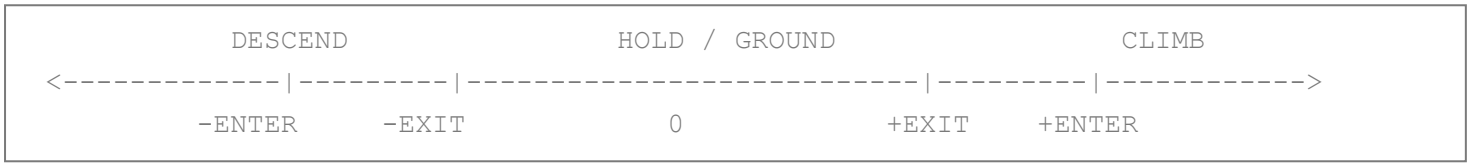
void UpdateDeltaAltitude(void)
{
    deltaAltitude = filteredAltitude - altHistory[histIndex];  // oldest entry
    altHistory[histIndex] = filteredAltitude;
    histIndex = (histIndex + 1) % DH_WINDOW_STEPS;
}
```

`previousAltitude` is no longer a single variable. The ring buffer replaces it.

17.2 Deadband with hysteresis

Do not use zero as the threshold. `if (deltaAltitude > 0)` makes noise toggle the state constantly.

A deadband alone is still not enough. If Δh hovers near the threshold, the state chatters. Use hysteresis: a larger threshold to *enter* CLIMB or DESCEND and a smaller one to *leave* it. Also require the entry condition on two consecutive updates.



17.3 GROUND vs. HOLD

When the sensor is not climbing or descending, the state depends on where it is:

- `|filteredAltitude| < GROUND_BAND_M` → GROUND
- otherwise → HOLD

17.4 Conceptual state logic

```
#define CLIMB_ENTER_M     /* from your measurement */
#define CLIMB_EXIT_M      /* smaller than CLIMB_ENTER_M */
#define DESCEND_ENTER_M   /* from your measurement */
#define DESCEND_EXIT_M    /* smaller than DESCEND_ENTER_M */
#define GROUND_BAND_M     /* e.g. a few tenths of a meter */
#define CONFIRM_STEPS     2

static VerticalState StationaryState(void)
{
    return (fabsf(filteredAltitude) < GROUND_BAND_M) ? GROUND : HOLD;
}

void UpdateVerticalState(void)
{
    UpdateDeltaAltitude();

    switch (verticalState)
    {
    case CLIMB:
        if (deltaAltitude < CLIMB_EXIT_M)
            verticalState = StationaryState();
        break;

    case DESCEND:
        if (deltaAltitude > -DESCEND_EXIT_M)
            verticalState = StationaryState();
        break;

    default:    // GROUND or HOLD
        if (deltaAltitude > CLIMB_ENTER_M)
        {
            descendCount = 0;
            if (++climbCount >= CONFIRM_STEPS) { verticalState = CLIMB; climbCount
= 0; }
        }
        else if (deltaAltitude < -DESCEND_ENTER_M)
        {
            climbCount = 0;
```

```
            if (++descendCount >= CONFIRM_STEPS) { verticalState = DESCEND;
descendCount = 0; }
        }
        else
        {
            climbCount = 0;
            descendCount = 0;
            verticalState = StationaryState();
        }
        break;
    }
}
```

The threshold values must come from your own measurements (Section 28).

---

18. Flight Dashboard

The LCD must display flight information, not raw debugging data. Required content:

- relative altitude (filtered)
- vertical flight state
- pressure
- temperature

A 16×2 LCD cannot show all of this at once, so implement two pages that alternate every 2 seconds (every 8 LCD updates).

Page 1 — Primary flight information

```
+----------------+
|ALT:   1.24 m   |
|STATE: CLIMB    |
+----------------+
```

Page 2 — Sensor information

```
+----------------+
|P: 97845.2 Pa   |
|T: 24.6 C       |
+----------------+
```

Fault screen (Section 20)

```
+----------------+
|BMP280 FAULT    |
|ID=0xFF EXP 0x58|
+----------------+
```

---

19. Task Rates

The BMP280 can be sampled much faster than a person can read an LCD. Updating the LCD on every sample wastes processor time and makes the display unreadable.

Task	Period	Rate
System timer tick	1 ms	1 kHz
BMP280 read + altitude + filter	50 ms	20 Hz
Vertical-state update	100 ms	10 Hz
LCD update	250 ms	4 Hz
LCD page change	2 s	0.5 Hz

The sensor produces data at about 25 Hz (Section 9), so 20 Hz sampling always reads a fresh value.

---

20. Sensor Fault Handling

The firmware must never hang because of the sensor, and must never display stale data as if it were valid.

- If the ID check fails at startup, enter `SYS_SENSOR_FAULT`.
- If any I²C transaction times out or is not ACKed while running, or the sanity check in Section 11.1 fails, enter `SYS_SENSOR_FAULT`.
- In `SYS_SENSOR_FAULT`:
  - the LCD shows the fault screen;
  - safety-event processing and the timer continue normally;
  - the firmware retries `BMP280_Init()` once per second. On success, it returns to `SYS_CALIBRATING`.

---

21. Non-Blocking Architecture and ISR Rules

Do not structure the program like this:

```
while (1)
{
    Read_BMP280();
    __delay_ms(50);
    Calculate_Altitude();
    __delay_ms(200);
    LCD_Update();
    __delay_ms(500);
}
```

This blocks the processor and destroys the timing architecture from previous labs.

The timer tells the main loop when work is due. The ISR stays short.

The ISR may:

```
Clear interrupt flags
Increment timing counters
Set task flags
Record urgent events
```

The ISR must not:

```
Update the LCD
Calculate altitude
Perform I2C transactions
Execute long loops
Use blocking delays
Perform floating-point calculations
```

```
Hardware Interrupt → Short ISR → Set Task Flag → Return → Main Loop → Perform Work
```

Timer ISR task generation (1 ms tick):

```
static uint16_t sensorCounter = 0, stateCounter = 0, lcdCounter = 0;

if (++sensorCounter >= 50)  { sensorCounter = 0; sensorTask = 1; }
if (++stateCounter  >= 100) { stateCounter  = 0; stateTask  = 1; }
if (++lcdCounter    >= 250) { lcdCounter    = 0; lcdTask    = 1; }
```

Use `uint16_t` counters so the periods can be changed later without overflow.

---

22. Program Structure

```
// ------------------------------------------------------
// Configuration Bits
// ------------------------------------------------------

// ------------------------------------------------------
// Includes and Definitions (pins, addresses, thresholds)
// ------------------------------------------------------

// ------------------------------------------------------
// Enumerations (SystemState, VerticalState)
// ------------------------------------------------------

// ------------------------------------------------------
// Global Variables
// ------------------------------------------------------

// ------------------------------------------------------
// I2C Functions (MSSP2, with timeouts)
// ------------------------------------------------------

// ------------------------------------------------------
// BMP280 Functions
// ------------------------------------------------------

// ------------------------------------------------------
// LCD: provided driver in lcd.c / lcd.h (do not copy it here)
// ------------------------------------------------------

// ------------------------------------------------------
// Calibration, Altitude and Filtering
// ------------------------------------------------------

// ------------------------------------------------------
// Vertical-State Processing
// ------------------------------------------------------

// ------------------------------------------------------
// Dashboard
// ------------------------------------------------------

// ------------------------------------------------------
// Timer / Interrupt Functions
// ------------------------------------------------------

// ------------------------------------------------------
```

```
// Initialization
// ------------------------------------------------

// ------------------------------------------------
// Main
// ------------------------------------------------
```

Do not put the entire lab inside `main()`. Separate hardware drivers from application logic.

---

23. Suggested Data

```
typedef enum { GROUND, CLIMB, HOLD, DESCEND } VerticalState;
typedef enum { SYS_CALIBRATING, SYS_RUNNING, SYS_SENSOR_FAULT } SystemState;

SystemState   systemState   = SYS_CALIBRATING;
VerticalState verticalState = GROUND;

float pressure;            // Pa
float temperature;         // deg C
float referencePressure;   // Pa, fixed after calibration

float rawAltitude;         // m
float filteredAltitude;    // m
float deltaAltitude;       // m over the Δh window

// Flags shared between the ISR and main must be volatile
volatile uint8_t sensorTask = 0;
volatile uint8_t stateTask  = 0;
volatile uint8_t lcdTask    = 0;
```

Use the enumeration names, not unexplained numbers like `verticalState = 2;`.

---

24. Main Program Architecture

```
void main(void)
{
    System_Init();          // oscillator, ANSEL/TRIS, LEDs and safety I/O from
earlier labs
    LCD_Init();             // one-time startup delays are acceptable here
    I2C_Init();
    Timer_Init();

    if (BMP280_Init())
        systemState = SYS_CALIBRATING;
    else
```

```
        systemState = SYS_SENSOR_FAULT;

    Enable_Interrupts();    // scheduler runs during calibration too

    while (1)
    {
        ProcessSafetyEvents();          // always runs, in every system state

        if (sensorTask)
        {
            sensorTask = 0;
            SensorTask();               // calibration, running, or fault retry
        }

        if (stateTask)
        {
            stateTask = 0;
            if (systemState == SYS_RUNNING)
                UpdateVerticalState();
        }

        if (lcdTask)
        {
            lcdTask = 0;
            UpdateFlightDashboard();    // shows calibration, dashboard, or fault
 screen
        }
    }
}
```

`SensorTask()` behaves differently depending on `systemState`:

`systemState`	What `SensorTask()` does
`SYS_CALIBRATING`	Read sensor, accumulate calibration samples (Section 14)
`SYS_RUNNING`	Read sensor → altitude → filter. On any error, enter fault
`SYS_SENSOR_FAULT`	Every 20th call (1 s), retry `BMP280_Init()`

This is an architectural example, not a complete solution. You implement the functions.

---

25. Required Development Procedure

Do not write the entire program and debug it all at once. Build it in stages. Do not move to the next stage until the current one works.

Stage 1 — Power and wiring

- J45 on 3V3. TVDD test pad measures about 3.3 V.
- PIC18F46K22 in the 40-pin socket, 28-pin socket empty, both mikroBUS sockets empty.
- BMP280 VCC from a 3V3 pin. LCD VDD from a 5V pin (or 3V3 if the module is rated for it).
- LCD R/W tied to GND.
- With power off, confirm I²C pull-ups exist (Section 4.3).
- With power on, SDA and SCL idle at about 3.3 V.
- Common ground everywhere.

Stage 2 — LCD bring-up

Add `lcd.c` and `lcd.h` to your project and define `_XTAL_FREQ` project-wide (Section 13.1). Then display a fixed message:

```
LCD_Init();
LCD_PrintLine(0, "HELLO ECE3301");
LCD_PrintLine(1, "LCD OK");
```

Adjust the contrast pot until the text is sharp. From now on, the LCD is your debug display.

Symptom	Likely cause
Top row of solid blocks, nothing else	LCD powered but not initialized: RS/E/D4–D7 wiring, or `_XTAL_FREQ` lower than the real clock
Nothing visible at all	Contrast pot, LCD power, or backlight wiring
Garbage or shifted characters	D4–D7 swapped or miswired, or R/W not grounded
Build error from `lcd.c` about `_XTAL_FREQ`	Macro not defined project-wide (Section 13.1, step 3)

Stage 3 — I²C identity

Read register 0xD0 and display it. It must show `0x58` (or `0x60`, see Section 8).

Stage 4 — Calibration coefficients

Read the 24 calibration bytes and display `dig_T1` and `dig_P1`. Neither should be 0x0000 or 0xFFFF.

Stage 5 — Temperature

Display compensated temperature. It must be physically reasonable for the lab. Touch the sensor and watch it rise.

Stage 6 — Pressure

Display compensated pressure. Expect roughly 95,000–102,000 Pa, depending on lab elevation and weather.

Stage 7 — Reference calibration

Implement the `SYS_CALIBRATING` state. Altitude must read approximately 0.00 m right after calibration.

Stage 8 — Altitude

Raise the sensor: altitude increases. Lower it: altitude decreases. If the sign is reversed, check the order of $P\$and\$P_{ref}$ in the equation.

Stage 9 — Filtering

Display raw and filtered altitude together. Compare stationary noise and response to movement for several α values.

Stage 10 — Vertical state

Measure stationary Δh (Section 28), set your thresholds, and verify GROUND → CLIMB → HOLD → DESCEND → GROUND.

Stage 11 — Dashboard

Implement the two alternating pages.

Stage 12 — Full integration and fault test

Run the complete scheduled system. Then disconnect the BMP280 SDA wire while running: the LCD must show the fault screen, and the rest of the firmware must keep running. Reconnect it and confirm the system recovers through recalibration.

---

26. Required System Behavior

Your completed Lab 8 system must:

1. Communicate with the BMP280 over I²C using MSSP2.

2. Verify the BMP280 chip ID before using any sensor data.
3. Obtain compensated temperature and pressure, compensating temperature first.
4. Establish a local reference pressure at startup using a non-blocking calibration state.
5. Calculate relative altitude from pressure.
6. Digitally filter the altitude.
7. Classify the vertical state as GROUND, CLIMB, HOLD or DESCEND.
8. Use a nonzero deadband with hysteresis so normal noise does not change the state.
9. Display altitude and vertical state on the LCD.
10. Display pressure and temperature on the LCD.
11. Run sensor acquisition, state update and LCD update at different rates.
12. Use timer/event scheduling rather than long blocking delays.
13. Perform I^2C transactions and LCD updates outside the ISR.
14. Use timeouts on all I^2C waits, and show a fault instead of hanging.
15. Be separated into logical functions.
16. Run continuously without requiring a reset.

---

27. Required Demonstration

Demonstrate the complete system to the instructor:

1. Calibration. Reset the system with the sensor stationary. The LCD must show the calibration screen with progress, then READY and approximately 0.00 m.
2. Stationary. Leave the sensor on the bench. Altitude stays reasonably stable and the state reports GROUND.
3. Climb. Raise the sensor about 0.5–1 m over 1–2 seconds. Altitude increases and the state shows CLIMB.
4. Hold. Keep the sensor still at the new height. After the altitude settles, the state shows HOLD.
5. Descend. Lower the sensor back to the bench over 1–2 seconds. Altitude decreases and the state shows DESCEND, then returns to GROUND.
6. Page 2. Show the pressure and temperature page.
7. Fault. Disconnect the BMP280 SDA wire. The LCD shows a fault and the system does not freeze. Reconnect it and show recovery.

Keep your hand off the sensor package during the demo. Body heat shifts the reading.

---

28. Engineering Challenge — Choose Your Thresholds From Data

There is no single correct threshold. Your thresholds must come from your hardware.

1. Add a third debug page that shows the minimum and maximum `deltaAltitude` since the page was selected:

```
+----------------+
|dH min: -0.04   |
|dH max:  0.05   |
+----------------+
```

2. Stationary measurement. Leave the sensor still on the bench for 30 seconds. Record the min and max Δh. Measure the same Δh signal your thresholds act on, not raw altitude.
3. Motion measurement. Perform the demonstration climb (0.5–1 m over 1–2 s) several times. Record the typical peak Δh.
4. Choose thresholds.
    - `CLIMB_ENTER_M` and `DESCEND_ENTER_M` should be comfortably above the stationary extremes (for example 1.5×) and comfortably below the motion peak (for example 0.5×).
    - Exit thresholds should be roughly half of the entry thresholds.
    - `GROUND_BAND_M` must be larger than the slow drift you observe over a few minutes.
5. If there is no gap between stationary noise and motion Δh, you must change the design. Which lever would you use, and what does it cost?
    - Larger α (smoother, but slower response)
    - Longer Δh window (better signal-to-noise, but slower detection)
    - Higher oversampling (already at maximum in this lab)

Be prepared to explain your numbers. This is the difference between writing software and designing an embedded system: the final value comes from the behavior of the physical hardware.

---

29. What You Must Submit

Submit your complete MPLAB X project to Canvas, including the provided `lcd.c` and `lcd.h` unmodified.

Your code must contain comments explaining:

- I²C (MSSP2) initialization and timeouts
- BMP280 initialization and register settings

- the burst read and compensation order
- reference-pressure calibration
- altitude calculation
- filtering and your chosen α
- vertical-state logic and your chosen thresholds, with the measured values that justify them
- task scheduling
- dashboard update: page switching and how you format each line
- fault handling

No formal report is required unless announced. Evaluation is based on your working demonstration and your ability to explain the system.

---

30. Common Problems

Symptom	Likely cause
ID reads 0xFF, or I²C times out	Wiring, missing pull-ups, wrong address (0x76 vs 0x77), PORTD left analog
ID reads 0x60	BME280 module — communication works
I²C or LCD stops working after adding hardware	A Click board in a mikroBUS socket is loading RD0–RD3 or RC3/RC4
Board will not program	`LVP = ON` missing from configuration bits, or a second PIC in the 28-pin socket
Readings fine, then sensor damaged or erratic	BMP280 powered from TVDD with J45 on 5V. Use the fixed 3V3 pin
Pressure is wildly wrong	Missing `(uint32_t)` casts, calibration bytes read in the wrong endian order, or pressure compensated before temperature
Pressure correct, altitude sign reversed	$P\$and\$P_{ref}$ swapped in the equation
Altitude ≈ 0 forever, even when moved	P_{ref} being updated after calibration

Symptom	Likely cause
Same value read twice in a row	BMP280 standby time too long for 20 Hz sampling
LCD shows a row of blocks	RS/E/D4–D7 wiring, or `_XTAL_FREQ` lower than the real clock
LCD flickers	Calling `LCD_Clear()` on every update
Leftover characters on the LCD	Using `LCD_Print` for the dashboard instead of `LCD_PrintLine`
LCD worked, stopped after adding I²C code	`I2C_Init()` wrote `TRISD` or `ANSELD` as a whole byte and undid `LCD_Init()`
System freezes when a wire is pulled	I²C wait loop without timeout
State flickers between CLIMB and HOLD	No hysteresis, or thresholds below the measured noise

---

31. What Your Drone Can Do After Lab 8

Your project now goes beyond basic microcontroller exercises:

```
Digital I/O
    ↓
Timing and Scheduling
    ↓
Safety State Machine
    ↓
Interrupt-Driven Events
    ↓
SPI → ICM-20948 Motion Sensing
    ↓
I2C → BMP280 Pressure Sensing
    ↓
Altitude Estimation and Filtering
    ↓
Vertical-State Classification
    ↓
Fault Detection
    ↓
LCD Flight Dashboard
```

The PIC now manages timing, responds to events, talks to two external sensors over two different buses, processes and filters measurements, classifies flight conditions, detects sensor failure, and presents information to the operator.

In the next lab, you will add ADC battery monitoring, so the dashboard can combine motion, altitude, battery condition and safety information into a more complete drone monitoring system.

Appendix A — Provided LCD Driver

These files are posted on Canvas. Add them to your project unmodified (Section 13.1).

A.1 `lcd.h`

```
/*
 * lcd.h - HD44780 16x2 character LCD driver (4-bit, write-only)
 *
 * ECE 3301 - Microcontrollers | Lab 8
 * Target: PIC18F46K22, MPLAB XC8
 *
 * Wiring (fixed by this driver):
 *     LCD RS     -> RD2
 *     LCD E      -> RD3
 *     LCD D4..D7 -> RD4..RD7
 *     LCD R/W    -> GND   (write-only: the busy flag is never read)
 *     LCD D0..D3 -> not connected
 *
 *     RD0 and RD1 are never touched (reserved for MSSP2 I2C).
 *
 * Requirements:
 *     - _XTAL_FREQ must be defined for the whole project and must equal
 *       the real CPU clock in Hz (see lcd.c).
 *     - Call these functions from main-loop code only, never from an ISR.
 *     - Every function blocks briefly (microseconds to a few ms).
 *       LCD_Init() blocks about 60 ms and is meant for startup only.
 */

#ifndef LCD_H
#define LCD_H

#include <stdint.h>

#define LCD_COLS  16u
#define LCD_ROWS  2u

/* HD44780 instruction codes used by this driver */
```

```
#define LCD_CMD_CLEAR         0x01u   /* clear display, cursor home (slow) */
#define LCD_CMD_HOME          0x02u   /* cursor home (slow)                */
#define LCD_CMD_ENTRY_INC     0x06u   /* increment address, no shift       */
#define LCD_CMD_DISPLAY_OFF   0x08u
#define LCD_CMD_DISPLAY_ON    0x0Cu   /* display on, cursor off, no blink  */
#define LCD_CMD_FUNC_4BIT_2L  0x28u   /* 4-bit bus, 2 lines, 5x8 font      */
#define LCD_CMD_SET_DDRAM     0x80u   /* OR with a DDRAM address           */

/* Initialize PORTD pins RD2..RD7 and the LCD. Call once at startup. */
void LCD_Init(void);

/* Send one instruction byte (RS = 0) and wait its execution time. */
void LCD_Command(uint8_t cmd);

/* Write one character at the current cursor position (RS = 1). */
void LCD_Char(char c);

/* Clear the whole display (takes 3 ms - do not use in periodic updates). */
void LCD_Clear(void);

/* Move the cursor. row: 0 = top, 1 = bottom. col: 0..15. */
void LCD_SetCursor(uint8_t row, uint8_t col);

/* Write a string starting at the current cursor position (no padding). */
void LCD_Print(const char *s);

/* Write a full line: exactly 16 characters, padded with spaces or
 * truncated. Use this for periodic dashboard updates - no flicker,
 * no leftover characters. */
void LCD_PrintLine(uint8_t row, const char *s);

#endif /* LCD_H */
```

A.2 `lcd.c`

```
/*
 * lcd.c - HD44780 16x2 character LCD driver (4-bit, write-only)
 *
 * ECE 3301 - Microcontrollers | Lab 8
 * Target: PIC18F46K22, MPLAB XC8
 *
 * See lcd.h for wiring and usage rules.
 *
 * Timing notes (HD44780U datasheet, VCC = 5 V):
 *   - Instruction execution: 37 us and 1.52 ms (clear/home) at 270 kHz.
 *     The LCD oscillator may be as slow as 190 kHz, which stretches these
 *     to about 53 us and 2.2 ms. R/W is grounded, so the busy flag cannot
```

```
 *     be read; this driver waits 60 us and 3 ms instead.
 *   - E high pulse >= 230 ns, data setup >= 80 ns before E falls.
 *     The LCD latches on the FALLING edge of E.
 *   - Power-on: wait > 15 ms after VCC reaches 4.5 V. This driver waits 50 ms.
 *   - The driver always initializes "by instruction" (0x3, 0x3, 0x3, 0x2),
 *     because the LCD's automatic power-on reset only works for supply
 *     rise times of 0.1-10 ms, which USB power does not guarantee.
 */

#include <xc.h>
#include <stdint.h>
#include "lcd.h"

#ifndef _XTAL_FREQ
#error "lcd.c: _XTAL_FREQ is not defined. Add _XTAL_FREQ=<your clock in Hz> under
Project Properties > XC8 Compiler > Preprocessing and messages > Define macros."
#endif

/* ---------------------------------------------------------------------
 * Pin definitions (must match lcd.h wiring)
 * --------------------------------------------------------------------- */
#define LCD_RS              LATDbits.LATD2
#define LCD_E               LATDbits.LATD3

#define LCD_KEEP_MASK       0x03u   /* RD0..RD1 belong to I2C - never touch */

/* ---------------------------------------------------------------------
 * Timing (see header comment for the reasoning)
 * --------------------------------------------------------------------- */
#define LCD_POWERUP_MS      50
#define LCD_SHORT_US        60      /* normal instructions and characters */
#define LCD_LONG_MS         3       /* clear display / return home        */
#define LCD_INIT_WAIT1_MS   5       /* after first 0x3 nibble  (> 4.1 ms)  */
#define LCD_INIT_WAIT2_US   150     /* after later init nibbles (> 100 us) */

/* ---------------------------------------------------------------------
 * Low level
 * --------------------------------------------------------------------- */

/* Put one nibble on D4..D7 and pulse E. RS must already be set.
 * Only RD4..RD7 change; RS (RD2), E (RD3), RD0 and RD1 are preserved. */
static void LCD_WriteNibble(uint8_t nibble)
{
    LATD = (uint8_t)((LATD & 0x0Fu) | (uint8_t)((nibble & 0x0Fu) << 4));

    LCD_E = 1;              /* data has been stable for > 80 ns already */
    __delay_us(1);          /* E high >= 230 ns                          */
```

```
    LCD_E = 0;              /* LCD latches the nibble on this edge       */
    __delay_us(1);          /* E cycle >= 500 ns                         */
}

/* Send a full byte as two nibbles, high nibble first. */
static void LCD_WriteByte(uint8_t value, uint8_t rs)
{
    LCD_RS = (rs != 0u) ? 1 : 0;    /* RS set before E rises (>= 40 ns) */
    LCD_WriteNibble((uint8_t)(value >> 4));
    LCD_WriteNibble((uint8_t)(value & 0x0Fu));
}

/* ---------------------------------------------------------------------
 * Public API
 * ------------------------------------------------------------------- */

void LCD_Command(uint8_t cmd)
{
    LCD_WriteByte(cmd, 0u);

    /* 0x01 (clear) and 0x02/0x03 (home) are the slow instructions */
    if (cmd <= 0x03u)
    {
        __delay_ms(LCD_LONG_MS);
    }
    else
    {
        __delay_us(LCD_SHORT_US);
    }
}

void LCD_Char(char c)
{
    LCD_WriteByte((uint8_t)c, 1u);
    __delay_us(LCD_SHORT_US);
}

void LCD_Init(void)
{
    /* RD2..RD7: digital outputs, driven low. RD0/RD1 left alone. */
    ANSELD &= LCD_KEEP_MASK;
    LATD   &= LCD_KEEP_MASK;
    TRISD  &= LCD_KEEP_MASK;

    /* Wait for the LCD supply to settle */
    __delay_ms(LCD_POWERUP_MS);
```

```
    /* Initialization by instruction (datasheet Figure 24, 4-bit) */
    LCD_RS = 0;
    LCD_WriteNibble(0x03u);
    __delay_ms(LCD_INIT_WAIT1_MS);
    LCD_WriteNibble(0x03u);
    __delay_us(LCD_INIT_WAIT2_US);
    LCD_WriteNibble(0x03u);
    __delay_us(LCD_INIT_WAIT2_US);
    LCD_WriteNibble(0x02u);             /* switch to 4-bit interface */
    __delay_us(LCD_INIT_WAIT2_US);

    /* From here on, every byte is sent as two nibbles */
    LCD_Command(LCD_CMD_FUNC_4BIT_2L);  /* 4-bit, 2 lines, 5x8 dots    */
    LCD_Command(LCD_CMD_DISPLAY_OFF);
    LCD_Command(LCD_CMD_CLEAR);
    LCD_Command(LCD_CMD_ENTRY_INC);
    LCD_Command(LCD_CMD_DISPLAY_ON);
}

void LCD_Clear(void)
{
    LCD_Command(LCD_CMD_CLEAR);
}

void LCD_SetCursor(uint8_t row, uint8_t col)
{
    uint8_t address;

    if (col >= LCD_COLS)
    {
        col = (uint8_t)(LCD_COLS - 1u);
    }

    /* 2-line display: line 1 starts at DDRAM 0x00, line 2 at 0x40 */
    address = (row == 0u) ? 0x00u : 0x40u;

    LCD_Command((uint8_t)(LCD_CMD_SET_DDRAM | (uint8_t)(address + col)));
}

void LCD_Print(const char *s)
{
    while (*s != '\0')
    {
        LCD_Char(*s);
        s++;
    }
}
```

```
void LCD_PrintLine(uint8_t row, const char *s)
{
    uint8_t i;

    LCD_SetCursor(row, 0u);

    for (i = 0u; i < LCD_COLS; i++)
    {
        if (*s != '\0')
        {
            LCD_Char(*s);
            s++;
        }
        else
        {
            LCD_Char(' ');      /* pad so old characters are overwritten */
        }
    }
}
```